DEFECT AUDIT & SECURITY ASSESSMENT

Software Defect & Vulnerability Report

PDF Vault & Digital Asset Storage System Codebase Audit

Document ID	BGR-PDFV-2026-V1	**Report Version**	1.0.0 (Pre-Release Audit)
Audit Date	October 20, 2026	**Lead Security Auditor**	Senior Application Security Engineer
Audit Scope	FastAPI Backend, React SPA, Storage Engine, Auth Subsystem	**Target Risk Posture**	Remediation Required Prior to Production Release
Classification	Internal Confidential / Security Advisory		

TOTAL DEFECTS	CRITICAL / HIGH	MEDIUM SEVERITY	LOW / USABILITY
7 Defects	**3 Items**	**3 Items**	**1 Item**
Identified in code audit	Immediate patch required	Sprint 1 remediation	Standard backlog item

1. Executive Summary & Audit Context

A rigorous static code analysis, architectural review, and security audit of the PDF Vault & Digital Asset Storage System was conducted across both backend (Python FastAPI) and frontend (React 18 SPA) codebases. While the core platform demonstrates solid engineering principles—including memory-hard Scrypt password hashing and binary magic-byte PDF validation—the audit revealed critical vulnerabilities and architectural edge cases that must be mitigated prior to public cloud launch.

> **Primary Audit Finding**
>
> The most significant vulnerability involves client-side only enforcement of the Starter Plan daily upload limit (localStorage), which allows users to bypass quota restrictions entirely via direct API calls or by clearing local browser cache. Additionally, fallback secret configurations and potential Windows file handle concurrency during 413 exceptions require immediate patching.

2. Defect Classification & Severity Standards

Severity Level	Definition & Impact	Resolution SLA
Critical (P1)	Direct security vulnerability, data loss, or business model compromise (e.g. bypassing monetization quotas).	Immediate (Within 24 Hours)
High (P2)	Severe flaw in security hygiene or data integrity impacting tenant isolation or file persistence.	Within 3 Business Days
Medium (P3)	Functional discrepancy, inconsistent API semantics, or platform-specific locking risks.	Within Current Sprint (1 Week)
Low (P4)	Minor usability inconsistency, documentation discrepancy, or cosmetic layout nuance.	Next Scheduled Release

3. Identified Defects Summary Matrix

Bug ID	Defect Title	Severity	Impacted Component	Remediation Status
BUG-001	Insecure Default JWT Secret Fallback	High (P2)	`backend/app/config.py`	Open (Patch Provided)
BUG-002	Client-Side Only Quota Enforcement (Free Tier Bypass)	Critical (P1)	`frontend/src/App.jsx` & Backend	Open (Patch Provided)
BUG-003	Nullable Document Tenancy Creating Orphaned Files	High (P2)	`backend/app/models.py`	Open (Patch Provided)
BUG-004	Shallow 5-Byte Magic Header Vulnerable to Polyglots	Medium (P3)	`backend/app/routers/documents.py`	Open (Patch Provided)

Bug ID	Defect Title	Severity	Impacted Component	Remediation Status
BUG-005	Frontend Enforces Mandatory Folder Contrary to Backend API	Medium (P3)	`frontend/src/App.jsx`	Open (Patch Provided)
BUG-006	Windows File Handle Lock Risk During 413 Exception	Medium (P3)	`backend/app/routers/documents.py`	Open (Patch Provided)
BUG-007	OAuth2 Form Parameter Inconsistency for Email Login	Low (P4)	`backend/app/auth/router.py`	Open (Patch Provided)

4. Detailed Defect Reports & Remediation Plans

4.1 BUG-001: Insecure Default JWT Secret Fallback

- **Severity / Priority:** High (P2) / Priority 1
- **Component:** `backend/app/config.py` (Line 9)
- **Description:** The Settings class initializes `auth_secret: str = 'change-this-development-secret'`. If the deployment environment lacks an explicit `.env` file or omits `AUTH_SECRET`, the API boots with a known public secret string.
- **Security Impact:** An attacker knowing this default development secret can forge arbitrary HMAC-SHA256 JWT tokens containing any user UUID, completely defeating multi-tenant isolation.
- **Recommended Remediation:** Enforce strict runtime validation: in production, require `AUTH_SECRET` to be provided and reject boot if the string is unchanged or shorter than 32 characters.

```
# Fix in backend/app/config.py:
from pydantic import model_validator

class Settings(BaseSettings):
    auth_secret: str

    @model_validator(mode='after')
    def validate_secret(self):
        if self.auth_secret == 'change-this-development-secret' or len(self.auth_secret) <
32:
            raise ValueError('A cryptographically secure AUTH_SECRET (min 32 chars) must be
configured.')
        return self
```

4.2 BUG-002: Client-Side Only Quota Enforcement (Free Tier Bypass)

- **Severity / Priority:** Critical (P1) / Priority 1
- **Component:** `frontend/src/App.jsx` (Lines 120–136)
- **Description:** The daily limit of 3 PDF uploads and 5 image uploads is evaluated exclusively inside the browser via `localStorage.getItem('pdf_vault_free_uploads_...')`. The FastAPI backend has zero awareness of user tiers or daily upload counts.
- **Business & Technical Impact:** Any user can clear their browser `localStorage`, open an Incognito tab, or issue direct cURL/Python API calls to upload unlimited files, completely bypassing the monetization model.
- **Recommended Remediation:** Implement server-side quota tracking. Add a `tier` column to the `User` model and a daily usage ledger in the database or Redis cache to enforce upload rate limits at the API gateway level.

```
# Server-side enforcement in backend/app/routers/documents.py:
@router.post('', status_code=201)
async def create_document(..., user: User = Depends(get_current_user), db: Session =
Depends(get_db)):
    today_start = datetime.now(timezone.utc).replace(hour=0, minute=0, second=0)
    daily_count = db.query(Document).filter(
        Document.user_id == user.id,
        Document.created_at >= today_start
    ).count()

    if user.plan == 'free' and daily_count >= 3:
        raise HTTPException(status_code=429, detail='Free tier daily upload limit (3 PDFs)
reached. Please upgrade to Pro.')
```

4.3 BUG-003: Nullable Document Tenancy Creating Orphaned Files

- **Severity / Priority:** High (P2) / Priority 2
- **Component:** `backend/app/models.py` (Line 12)
- **Description:** The Document model defines `user_id: Mapped[str | None] = mapped_column(ForeignKey('users.id'), nullable=True)`. However, `list_documents`, `download_document`, and `remove_document` query `Document.user_id == user.id`.
- **Impact:** Any document inserted with a NULL `user_id` becomes permanently invisible and un-deletable through the user interface, resulting in orphaned storage consumption.
- **Recommended Remediation:** Alter `user_id` to `nullable=False` across the schema and add database-level cascade constraints.

4.4 BUG-004: Shallow 5-Byte Magic Header Vulnerable to Polyglots

- **Severity / Priority:** Medium (P3) / Priority 2
- **Component:** `backend/app/routers/documents.py` (Line 72)
- **Description:** The validation checks only whether the first 5 bytes match `b'%PDF-'`. Malicious actors can craft polyglot files (e.g., an executable binary or HTML payload prefixed with `%PDF-`) that evade this check while posing potential risks in downstream viewers.
- **Recommended Remediation:** Integrate a robust PDF parser (e.g. `pypdf.PdfReader` header and trailer validation) to ensure document syntax validity.

```
# Enhanced validation:
import pypdf

def validate_pdf_structure(file_path):
    try:
        reader = pypdf.PdfReader(file_path)
        if len(reader.pages) == 0:
            raise ValueError('PDF has zero pages.')
    except Exception as e:
        raise HTTPException(415, 'Corrupted or invalid PDF structure.')
```

4.5 BUG-005: Frontend Enforces Mandatory Folder Contrary to Backend API

- **Severity / Priority:** Medium (P3) / Priority 3
- **Component:** `frontend/src/App.jsx` (Lines 116–118)
- **Description:** The frontend upload handler throws an error if no folder is active: `if (!folderId) throw new Error('Open a folder before uploading a PDF.')`. Conversely, the backend API explicitly supports root-level documents (`folder_id: str = Form('')`).
- **Impact:** New users who have not yet created folders are blocked from uploading PDFs, causing severe UX friction.
- **Recommended Remediation:** Remove the mandatory folder check from the frontend or provide an 'All Documents / Uncategorized' root folder by default.

4.6 BUG-006: Windows File Handle Lock Risk During 413 Exception

- **Severity / Priority:** Medium (P3) / Priority 3
- **Component:** `backend/app/routers/documents.py` (Lines 66–84)
- **Description:** When a file exceeds 25MB, an `HTTPException(413)` is raised inside `with target.open('wb') as out:`. On Windows operating systems, calling `target.unlink(missing_ok=True)` in the outer `except` block can occasionally trigger a `PermissionError: [WinError 32]` if the file handle has not finished closing.
- **Recommended Remediation:** Ensure the `with` block has exited cleanly before attempting file deletion:

```
# Safe handle cleanup:
exceeded = False
with target.open('wb') as out:
    while chunk := await file.read(1024 * 1024):
        size += len(chunk)
        if size > limit:
            exceeded = True
            break
        out.write(chunk)

if exceeded:
    target.unlink(missing_ok=True)
    raise HTTPException(413, f'Maximum upload size is {settings.max_upload_mb} MB.')
```

4.7 BUG-007: OAuth2 Form Parameter Inconsistency for Email Login

- **Severity / Priority:** Low (P4) / Priority 4
- **Component:** `backend/app/auth/router.py` (Lines 51–59)
- **Description:** The login route uses `OAuth2PasswordRequestForm`, which designates the credential input as `username`. However, the system authenticates against user email: `User.email == form.username.strip().lower()`.
- **Impact:** External API consumers expecting standard JSON body `{ email, password }` encounter confusion when having to supply form-urlencoded data with an `email` value placed in a `username` key.
- **Recommended Remediation:** Support both standard OAuth2 form login (for Swagger UI) and a dedicated JSON endpoint `POST /api/v1/auth/login-json` accepting `{ "email": "...", "password": "..." }`.

5. Defect Remediation Roadmap & Sprint Allocation

The engineering squad will execute patches according to the following phased schedule:

Sprint Phase	Target Defects	Estimated Effort	Target Completion Date
Hotfix Sprint (Immediate)	BUG-001 (JWT Secret), BUG-002 (Server Quota Enforcement)	8 Engineering Hours	October 22, 2026
Sprint 1 (Current)	BUG-003 (Orphan Tenancy), BUG-005 (Folder UX), BUG-006 (File Handle Lock)	12 Engineering Hours	October 25, 2026

Sprint Phase	Target Defects	Estimated Effort	Target Completion Date
Sprint 2 (Enhancement)	BUG-004 (Deep PDF Validation), BUG-007 (Dual Auth Form/JSON)	10 Engineering Hours	October 29, 2026

6. Security Audit & Defect Governance Sign-Off

By signing below, the Technical Lead and Security Auditor confirm the findings and authorize the remediation schedule:

Lead Security Auditor	Technical Project Lead
Signature: _____	Signature: _____
Name: Senior Application Security Engineer	Name: Solutions Architect / Tech Lead
Title: Lead Auditor & Security Specialist	Title: VP of Software Engineering
Date: October 20, 2026	Date: October 20, 2026